

Project Documentation

End-to-End ETL Pipeline using PySpark, Apache Airflow, Docker, and SQL Server

Digital Egypt Pioneers Initiative (DEPI)

Microsoft Data Engineering Track

Prepared by:

Marwan Ahmed

Team Members:

Kareem Adel Abdelhamid

Abdulrahman Elsayed Alshobaki

Hamza Mahmoud Negm

ali yasser ali sayed

Instructor:

Mohamed Roshdy

Project overview

Overview

This project presents an end-to-end ETL (Extract, Transform, Load) pipeline developed using PySpark, Apache Airflow, Docker, and Microsoft SQL Server as part of the Digital Egypt Pioneers Initiative (DEPI) under the Microsoft Data Engineering Track.

The pipeline processes the Brazilian Olist E-commerce Dataset by extracting raw CSV files, transforming the data through cleaning, validation, joins, and business aggregations using PySpark, and loading the processed results into a SQL Server data warehouse.

Apache Airflow is used to orchestrate and automate the ETL workflow, while Docker provides a containerized environment for deployment and reproducibility. The generated analytical datasets support business reporting by providing insights such as sales by state, sales by product category, and overall retail analytics.

This project demonstrates modern data engineering practices, including workflow automation, scalable data processing, containerization, and data warehousing, simulating a real-world production ETL pipeline.

Objectives

- Design and implement an end-to-end ETL pipeline.
- Extract raw e-commerce data from multiple CSV datasets.
- Transform data using PySpark through cleaning, validation, and aggregation.
- Load processed analytical data into Microsoft SQL Server.
- Automate the ETL workflow using Apache Airflow.
- Containerize the project using Docker.
- Generate business analytics for reporting and decision-making.
- Demonstrate modern data engineering best practices.

Scope

- Data Source:
Large e-commerce order dataset (CSV format).
- Processing Tool

PySpark for distributed data processing.

- Data Operations:

Handling missing or null values.

Removing duplicate records.

Joining multiple datasets.

Data aggregation and summarization.

Sales analysis by category.

Sales analysis by state.

- Output:

Processed analytical tables loaded into Microsoft SQL Server for reporting and business analysis.

The project covers the complete ETL lifecycle, including data extraction, transformation, loading data into Microsoft SQL Server, and workflow orchestration using Apache Airflow in a Dockerized environment. The implemented pipeline simulates a real-world data engineering workflow and demonstrates industry best practices for scalable ETL development.

Project timeline

Phase	Task	Start Date	Duration
Phase 1	Data Collection	1/5/2026	7 days
Phase 2	Data Cleaning	8/5/2026	14 days
Phase 3	ETL Development	22/5/2026	10 days
Phase 4	Deployment	1/6/2026	5 days
Phase 5	Testing	6/6/2026	7 days
Phase 6	Documentation	13/6/2026	7 days

Milestones

Milestone 1: Data Collection & Exploration (May 2026)

Collection of the Olist Brazilian E-commerce dataset and initial data exploration to understand the dataset structure, relationships, and data quality issues.

Milestone 2: Data Cleaning & Transformation (May 2026)

Development of PySpark transformation scripts to clean the data, handle missing values, remove duplicate records, join datasets, and generate analytical outputs.

Milestone 3: ETL Pipeline Development (Late May 2026)

Implementation of a complete ETL pipeline using PySpark to extract, transform, and prepare data for loading into Microsoft SQL Server.

Milestone 4: SQL Server Integration & Workflow Automation (June 2026)

Loading transformed datasets into Microsoft SQL Server and automating the ETL workflow using Apache Airflow within a Dockerized environment.

Milestone 5: Testing & Validation (June 2026)

Verification of data accuracy, ETL pipeline execution, SQL Server integration, and workflow automation through comprehensive testing.

Milestone 6: Documentation & Final Presentation (June 2026)

Completion of the project documentation, GitHub repository preparation, presentation materials, and final project demonstration.

Deliverables

- Raw Dataset

Collection of the original e-commerce dataset used as the input for the ETL process.

- Processed Analytical Dataset

Processed analytical datasets generated after data cleaning, transformation, joining, and aggregation using PySpark. The datasets are prepared for loading into Microsoft SQL Server and business reporting.

- PySpark ETL Scripts

Well-structured and documented PySpark scripts used to implement the extraction, transformation, and loading (ETL) processes.

- ETL Pipeline Implementation

A complete ETL pipeline that automates the workflow from raw data extraction through transformation and loading into Microsoft SQL Server using Apache Airflow.

- Project Documentation

A comprehensive report including project planning, system design, implementation details, testing process, and project outcomes.

- GitHub Repository

A version-controlled GitHub repository containing the complete source code, project documentation, screenshots, Docker configuration files, SQL scripts, and all required project resources.

- Final Presentation

A PowerPoint presentation summarizing the project objectives, methodology, system architecture, implementation, challenges, and final results.

Resource allocation

1-Human resource

Data Engineer(s):

Responsible for building the ETL pipeline using PySpark, implementing data cleaning logic, and ensuring scalability.

Data Analyst:

Responsible for exploring the dataset, identifying data quality issues, and validating the cleaned data.

Documentation Specialist:

Responsible for writing the project report, preparing documentation, and organizing deliverables.

2-Technical Resources

- Programming Language: Python
- Framework: PySpark (Apache Spark)
- Workflow Orchestration: Apache Airflow
- Containerization: Docker
- Database: Microsoft SQL Server
- Development Environment: Visual Studio Code
- Version Control: Git & GitHub
- Data Storage: CSV Files

Team Members and Responsibilities

Marwan ahmed	• Responsible for: • • Project planning • ETL pipeline development using PySpark • Data extraction and transformation • SQL Server integration • Apache Airflow workflow orchestration • Docker containerization • Testing and validation • GitHub repository management • Project documentation
Kareem adel	Responsible for: • ERD design • Database schema design • System requirements • Database tables documentation
Abdelrahman elsayed	Responsible for: • Documentation support • Project review
Hamza Mahmoud	Responsible for: • Documentation support • Testing assistance
Ali Yasser	Responsible for: • Presentation preparation • Documentation formatting support

Risk Assessment & Mitigation Plan

Risk 1: Large Dataset Handling Issues

Description:

The e-commerce dataset may become too large to process efficiently on limited local computing resources, resulting in slower execution and higher memory consumption.

Mitigation:

Optimize PySpark transformations, reduce unnecessary data movement, use partitioning when appropriate, and validate performance during each development stage. If required, the pipeline can be executed in a distributed environment.

Risk 2: Data Quality Issues

Description:

The raw dataset may contain missing values, duplicate records, inconsistent formats, or invalid data that could affect the accuracy of the analytical results.

Mitigation:

Apply comprehensive data cleaning techniques including missing value handling, duplicate removal, data validation, and format standardization before loading the processed data into SQL Server.

Risk 3: PySpark Development Challenges

Description:

Developing and optimizing distributed data processing workflows using PySpark may introduce implementation and debugging challenges during the project.

Mitigation:

Follow PySpark best practices, review the official documentation, test each transformation independently, and validate intermediate outputs throughout the ETL process.

Risk 4: Time Constraints

Description:

Limited project time may affect the completion of development, testing, documentation, and final presentation.

Mitigation:

Follow a clear project schedule, assign responsibilities effectively among team members, monitor progress regularly, and prioritize critical development tasks.

Risk 5: Integration and Workflow Automation Issues

Description:

Integrating PySpark, SQL Server, Docker, and Apache Airflow into a single automated ETL workflow may introduce configuration or compatibility issues.

Mitigation:

Test each ETL stage separately before integrating the complete workflow, verify Docker services, validate Airflow DAG execution, and perform end-to-end testing before deployment.

Risk 6: Data Loss or Processing Errors

Description:

Unexpected failures during data transformation or loading may result in incomplete or incorrect analytical outputs.

Mitigation:

Perform continuous testing, validate processed datasets after every ETL stage, maintain backup copies of the raw datasets, and verify SQL Server outputs before generating reports.

KPIs

1. Data Processing Time

Description:

Measures the total time required to complete the ETL pipeline, including data extraction, transformation, and loading.

Target:

Complete the full ETL process within an acceptable execution time while maintaining data accuracy.

—

2. Data Quality Improvement

Description:

Measures the percentage of data quality issues resolved, including missing values, duplicates, and inconsistent records.

Target:

Achieve at least 95% clean and validated data before loading into SQL Server.

—

3. Pipeline Success Rate

Description:

Measures the percentage of successful ETL pipeline executions without runtime failures.

Target:

Maintain a success rate of 95% or higher.

—

4. Data Loading Accuracy

Description:

Measures the accuracy of loading transformed data into SQL Server without missing or corrupted records.

Target:

Achieve 100% successful data loading with verified record counts.

—

5. System Performance

Description:

Evaluates the efficiency of the ETL pipeline while processing multiple datasets using PySpark and Docker.

Target:

Ensure stable performance with efficient resource utilization throughout execution.

—

6. Data Consistency

Description:

Measures whether the transformed datasets maintain consistent formats, standardized values, and valid relationships across all tables.

Target:

Maintain 100% consistency in processed analytical datasets.

Requirements Gathering

Stakeholder analysis

- Project Team (Developers & Engineers):
Need a clear and structured pipeline to implement data cleaning efficiently.
- Data Analysts:
Require clean, accurate, and structured data for analysis and reporting.
- Instructors / Evaluators:
Expect a well-documented project with proper implementation and demonstration.
- End Users (Optional):
May use the cleaned dataset for insights and decision-making.

User Stories & Use Cases

User stories

- As a Data Engineer, I want to automate the ETL pipeline so that data can be processed efficiently.
- As a Data Analyst, I want clean and structured data so that I can generate reliable business insights.
- As an Instructor, I want to evaluate a complete end-to-end ETL solution demonstrating data engineering best practices.

Use cases

Use Case 1: Data Extraction

The system extracts raw retail datasets from multiple CSV files.

Use Case 2: Data Transformation

The system cleans, validates, joins, and aggregates the datasets using PySpark.

Use Case 3: Data Loading

The processed analytical tables are loaded into Microsoft SQL Server.

Use Case 4: Workflow Automation

Apache Airflow schedules and executes the ETL pipeline automatically.

Functional Requirements

- The system should load large datasets into PySpark
- The system should detect and handle missing values
- The system should remove duplicate records
- The system should standardize data formats (dates, prices, etc.)
- The system should validate and clean incorrect data entries
- The system should output cleaned data in a structured format
- The system should allow re-running the pipeline when needed
- The system should load processed data into Microsoft SQL Server.
- The system should support workflow automation using Apache Airflow.

Non-Functional Requirements

- Performance:
The system should process large datasets efficiently using PySpark.
- Scalability:
The system should handle increasing data sizes without performance degradation.
- Reliability:
The system should produce consistent and accurate outputs.

- Usability:
The system should be easy to run and understand by users.
- Maintainability:
The code should be clean, modular, and easy to update.
- Data integrity
The system should maintain data integrity throughout the ETL process and prevent data corruption.

System Analysis & Design

Problem Statement & Objectives

In real-world data engineering environments, raw datasets—especially in e-commerce—often suffer from poor data quality. Common issues include missing values, duplicate records, inconsistent formats, and invalid entries. These problems significantly affect data reliability and make it difficult to perform accurate analysis.

Additionally, handling large-scale datasets using traditional tools can be inefficient and time-consuming. Therefore, there is a need for a scalable and automated system that can efficiently clean and process large datasets.

Objectives

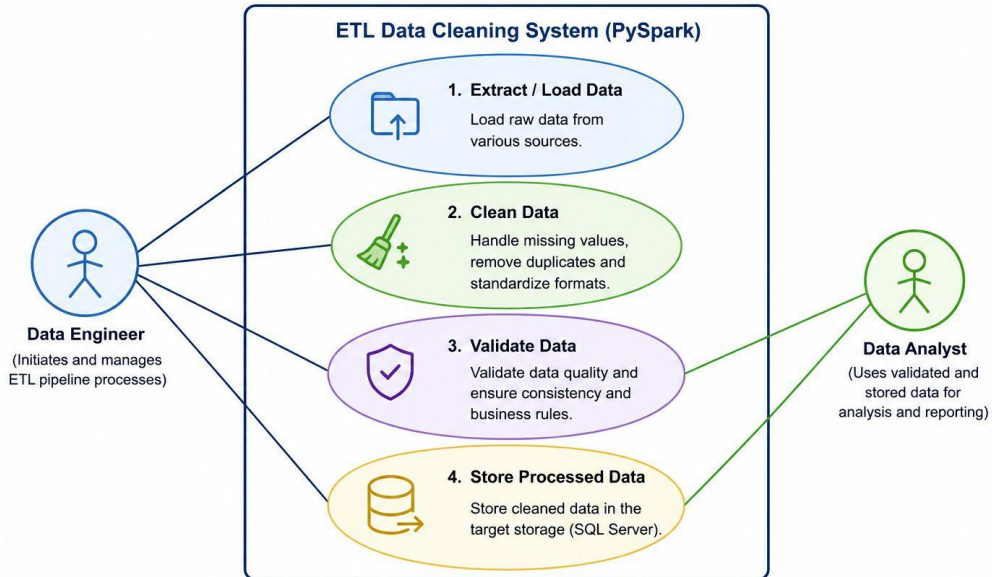
- * Build a scalable end-to-end ETL pipeline using PySpark.
- * Extract data from multiple CSV datasets.
- * Clean, validate, and transform raw e-commerce data.
- * Load processed analytical data into Microsoft SQL Server.
- * Automate the ETL workflow using Apache Airflow.
- * Containerize the application using Docker.
- * Generate business-ready analytical datasets for reporting.

Use case diagram

The following use case diagram illustrates the interaction between the Data Engineer and the ETL pipeline components throughout the extraction, transformation, loading, and automation processes.

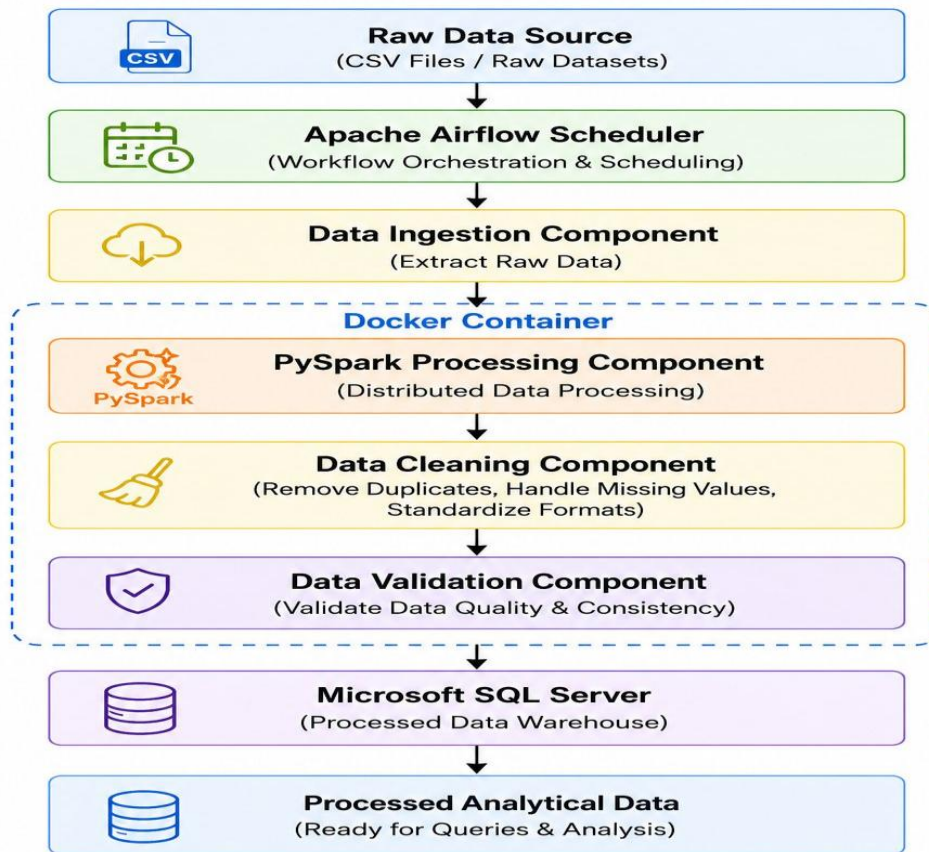
Use Case Diagram

The following use case diagram illustrates the interaction between the Data Engineer and the ETL pipeline components throughout the extraction, transformation, loading, and automation processes.



Software Architecture Diagram

Software Architecture Diagram



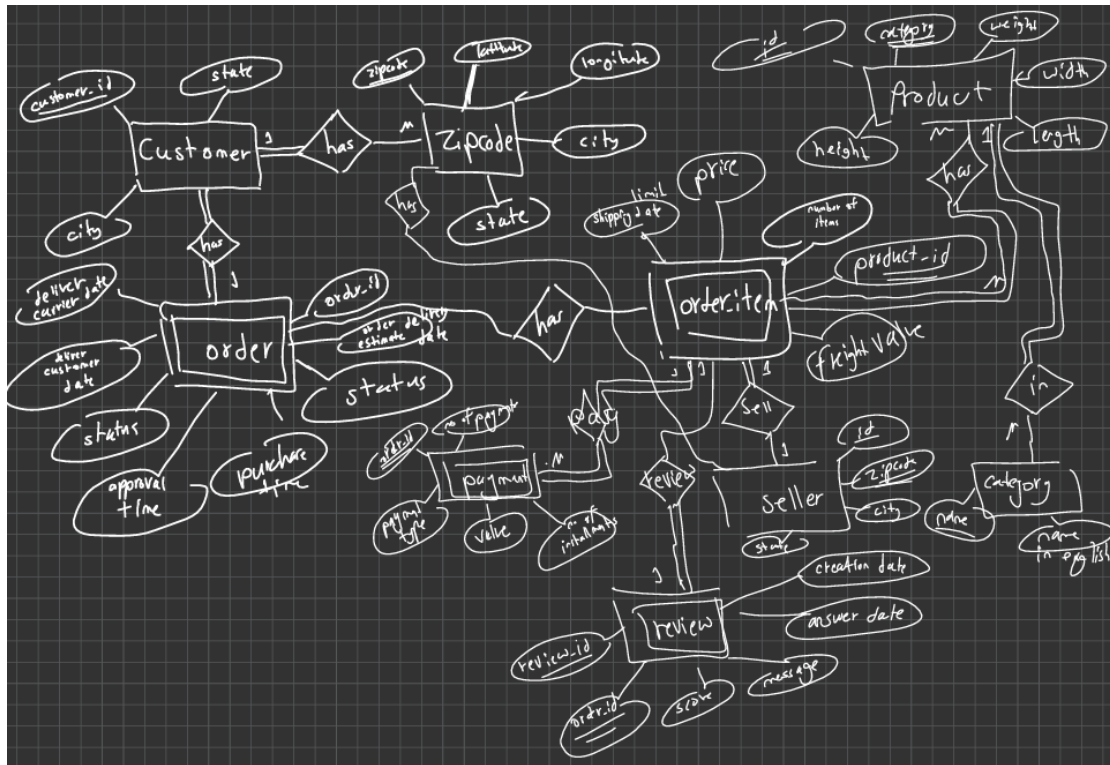
System Requirements

System requirements:-

- * Every customer must have a unique_id and a city and state and zip code.
- * zip code refers to lat, lng, city, state
- * order item has : number of items, and the product name and the seller and shipping limit date and price and freight value and payment, review
- * every payment must have order id, no. of payments, payment types, value, number of installments
- * review has review_id, order_id, score, comment title, message, creation date, answer date
- * every order has id, customer, status, purchase time, approval time, deliver carrier time, deliver customer date, order estimated deliver date, order item
- * product has id, category name, weight, length, height, width
- * sellers has id, zip code, city, state
- * category has name in Port., English.

Entities: Customer, Zipcode, Order, Order Item,
Payment, Review, Product, Seller,
Category

ERD DIAGRAM



TABLES

1-Customer:

Customer_ID PK, STATE, CITY, Zipcode FK

2-Zipcode:

Zipcode PK, latitude, longitude, city,state

3-Product:

Id PK,category FK, weight, width, length, height

4-Category:

Name PK, name in English

5- Seller:

Id PK, zipcode FK, city, state

6- Order:

Order_id PK, order estimate delivery date, order status, order purchase time, order approval time, order deliver customer date, deliver carrier date, customer FK

7-Payment:

Number of payments, Order_item_id FK, payment type, value, no of installments

8-Review

Review_id pk, order_id FK, score, message, answer date, creation date, order_item_id FK

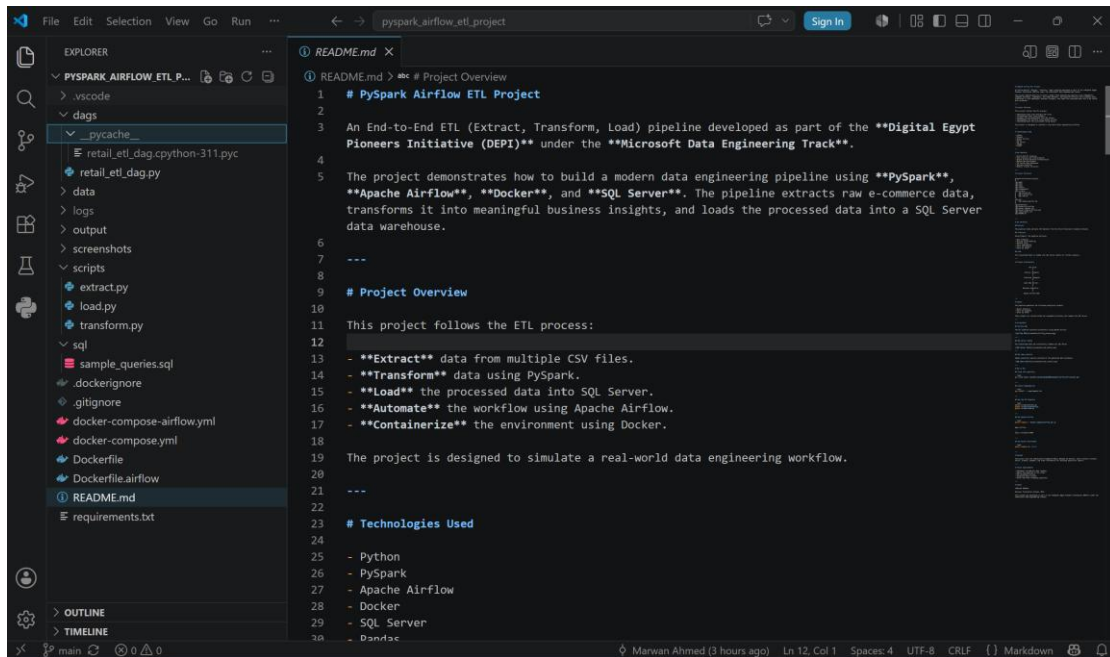
9- Order_item:

Order_item ID, Price, shipping date limit, number of items, product_id, flight value, order_id FK, Seller_id FK

Implementation & Results

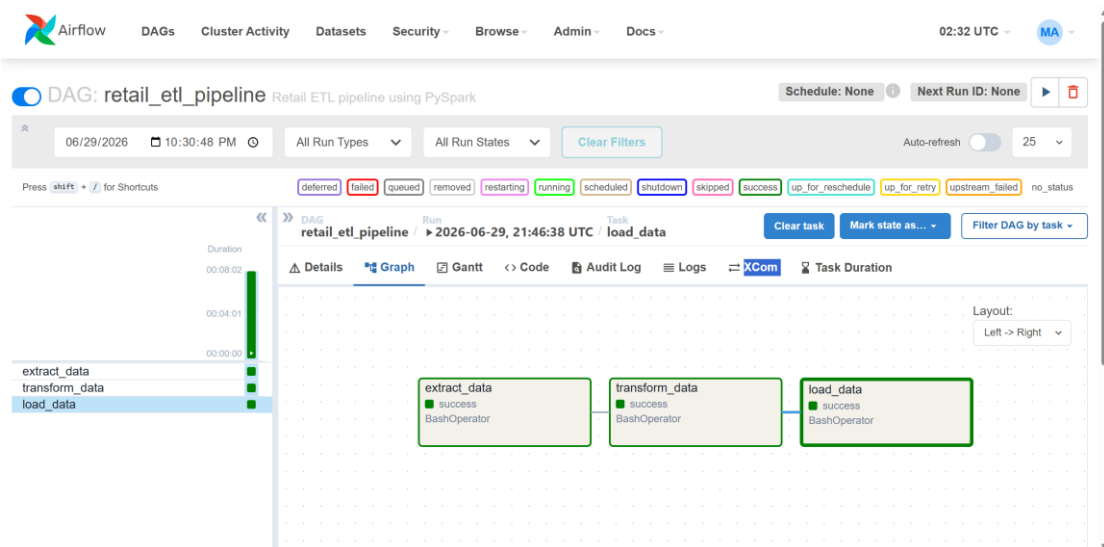
1- Project structure

Figure 1: Project folder structure in VS Code.d



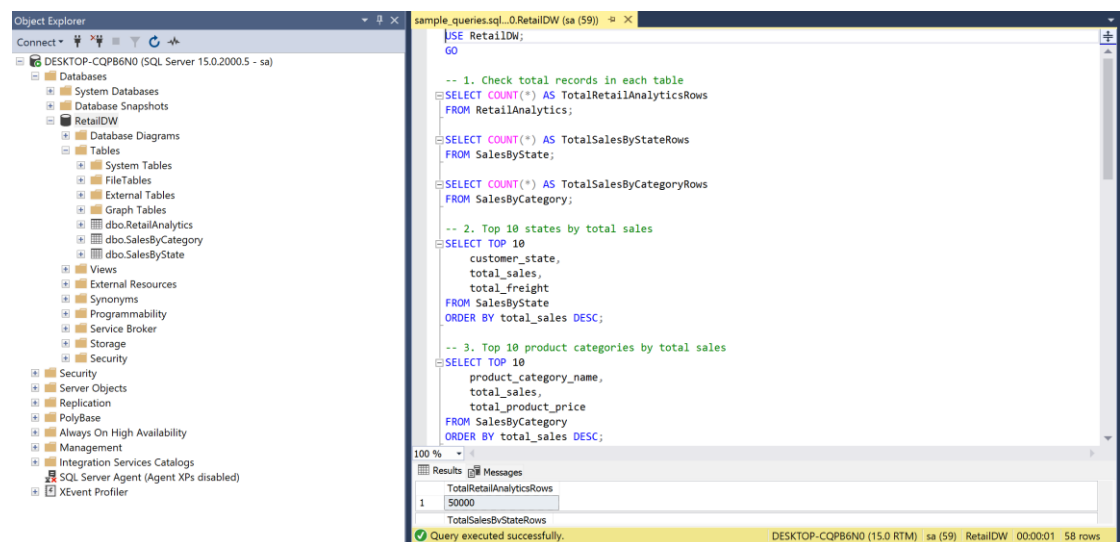
Air flow dag success

Figure 2: Successful execution of the ETL pipeline using Apache Airflow.



Sql server database

Figure 3: SQL Server database tables generated after loading processed data.



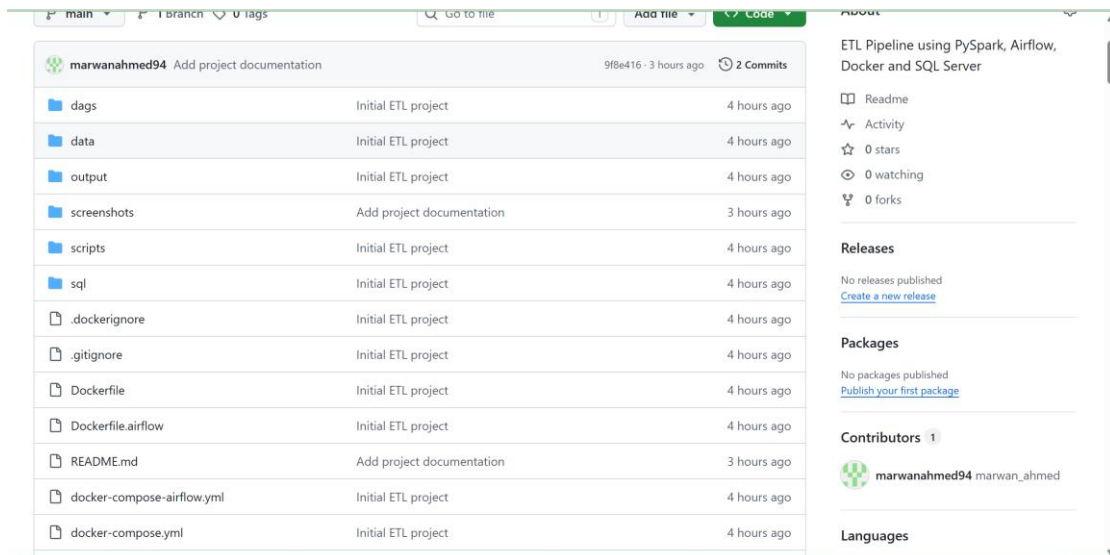
Sql query results

Figure 4: Sample SQL analytical queries executed on the processed data.

sample_queries.sql...0.RetailDW (sa (59))			
100 %			
Results Messages			
TotalRetailAnalyticsRows			
1	50000		
TotalSalesByStateRows			
1	27		
TotalSalesByCategoryRows			
1	74		
1	customer_state	total_sales	total_freight
1	SP	7597209.660000001	718714.579999999
2	RJ	2769347.44	305589.309999999
3	MG	2326151.64	270853.459999999
4	RS	1147277	135522.74
5	PR	1064603.99	117851.68
6	BA	797410.36	100156.68
7	SC	786343.71	89660.2600000001
8	GO	513879	53114.98
1	product_category_name	total_sales	total_product_price
1	cama_mesa_banho	1712553.67	1036988.679999999
2	beleza_saude	1657373.12	1258546.369999999
3	informatica_acessorios	1585330.45	911954.319999999
4	moveis_decoracao	1430176.39	729762.489999999
5	relogios_presentes	1429216.68	1205005.68
6	esporte_lazer	1392127.56	988048.969999999
7	utilidades_domesticas	1094758.13	632248.66
8	automotivo	852294.33	592720.11
TotalSales		TotalFreight	
1	9038855.79999997	1002497.109999995	
AverageDeliveryDays			
1	12.418162144505		

GitHub Repository

Figure 5: GitHub repository containing the project source code and documentation.

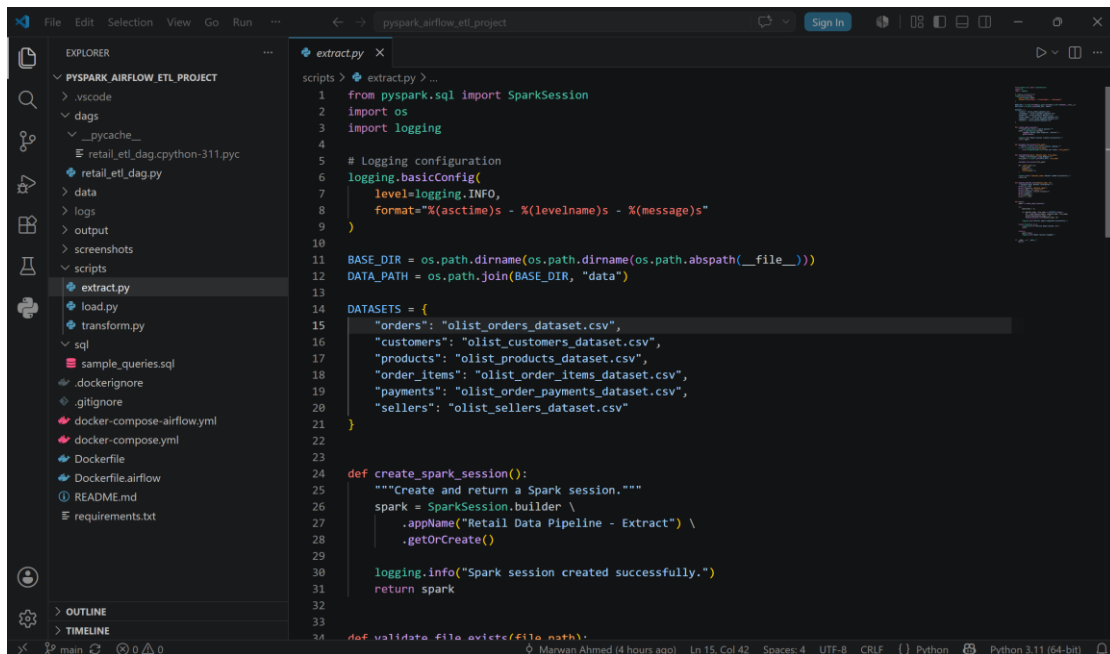


GitHub Repository:

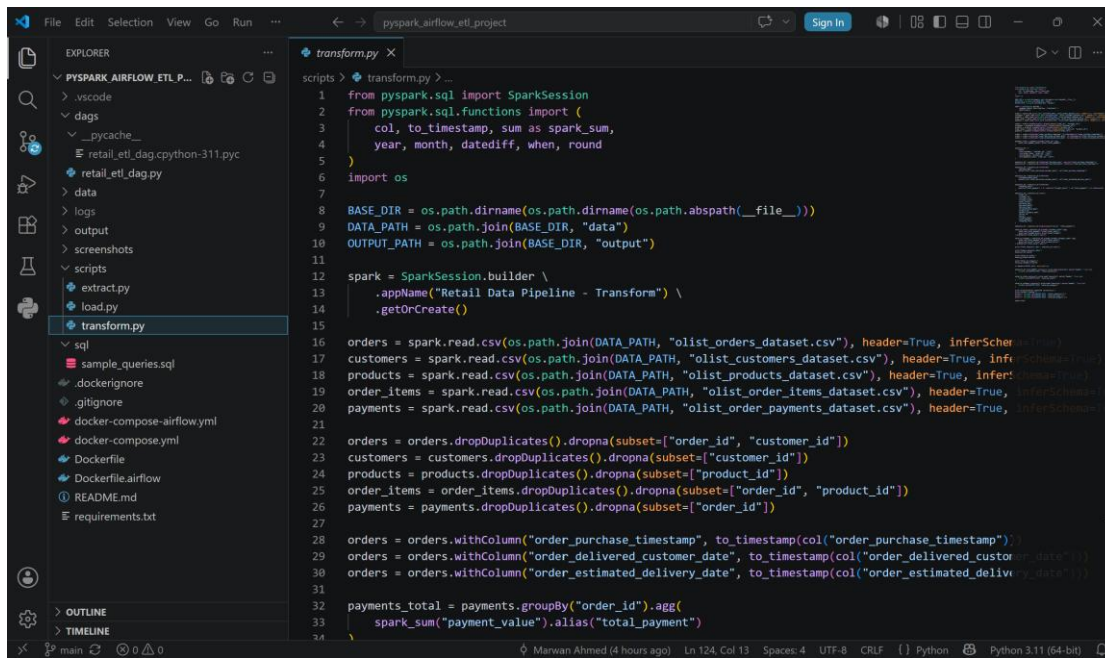
<https://github.com/marwanahmed94/pyspark-airflow-etl-project>

code snippets

extract phase

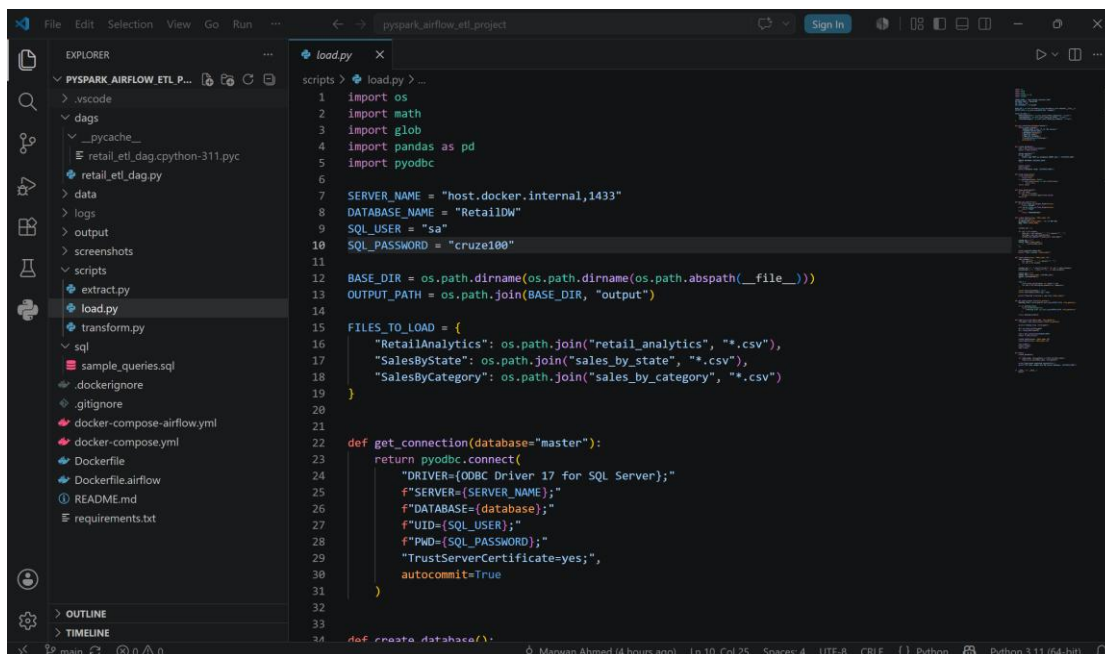


Transform phase



```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import (
3     col, to_timestamp, sum as spark_sum,
4     year, month, datediff, when, round
5 )
6 import os
7
8 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
9 DATA_PATH = os.path.join(BASE_DIR, "data")
10 OUTPUT_PATH = os.path.join(BASE_DIR, "output")
11
12 spark = SparkSession.builder \
13     .appName("Retail Data Pipeline - Transform") \
14     .getOrCreate()
15
16 orders = spark.read.csv(os.path.join(DATA_PATH, "olist_orders_dataset.csv"), header=True, inferSchema=True)
17 customers = spark.read.csv(os.path.join(DATA_PATH, "olist_customers_dataset.csv"), header=True, inferSchema=True)
18 products = spark.read.csv(os.path.join(DATA_PATH, "olist_products_dataset.csv"), header=True, inferSchema=True)
19 order_items = spark.read.csv(os.path.join(DATA_PATH, "olist_order_items_dataset.csv"), header=True, inferSchema=True)
20 payments = spark.read.csv(os.path.join(DATA_PATH, "olist_order_payments_dataset.csv"), header=True, inferSchema=True)
21
22 orders = orders.dropDuplicates().dropna(subset=["order_id", "customer_id"])
23 customers = customers.dropDuplicates().dropna(subset=["customer_id"])
24 products = products.dropDuplicates().dropna(subset=["product_id"])
25 order_items = order_items.dropDuplicates().dropna(subset=["order_id", "product_id"])
26 payments = payments.dropDuplicates().dropna(subset=["order_id"])
27
28 orders = orders.withColumn("order_purchase_timestamp", to_timestamp(col("order_purchase_timestamp")))
29 orders = orders.withColumn("order_delivered_customer_date", to_timestamp(col("order_delivered_customer_date")))
30 orders = orders.withColumn("order_estimated_delivery_date", to_timestamp(col("order_estimated_delivery_date")))
31
32 payments_total = payments.groupBy("order_id").agg(
33     spark_sum("payment_value").alias("total_payment")
34 )
```

Load phase



```
1 import os
2 import math
3 import glob
4 import pandas as pd
5 import pyodbc
6
7 SERVER_NAME = "host.docker.internal,1433"
8 DATABASE_NAME = "RetailDW"
9 SQL_USER = "sa"
10 SQL_PASSWORD = "cruze100"
11
12 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
13 OUTPUT_PATH = os.path.join(BASE_DIR, "output")
14
15 FILES_TO_LOAD = {
16     "RetailAnalytics": os.path.join("retail_analytics", "*.csv"),
17     "SalesByState": os.path.join("sales_by_state", "*.csv"),
18     "SalesByCategory": os.path.join("sales_by_category", "*.csv")
19 }
20
21
22 def get_connection(database="master"):
23     return pyodbc.connect(
24         "DRIVER={ODBC Driver 17 for SQL Server};"
25         f"SERVER={SERVER_NAME};"
26         f"DATABASE={database};"
27         f"UID={SQL_USER};"
28         f"PWD={SQL_PASSWORD};"
29         "TrustServerCertificate=yes;"
30         "autocommit=True"
31     )
32
33
34 def create_database():
```

Conclusion

This project successfully demonstrates the design and implementation of an end-to-end ETL (Extract, Transform, Load) pipeline using modern data engineering technologies, including PySpark, Apache Airflow, Docker, and Microsoft SQL Server.

The developed solution efficiently extracts raw e-commerce datasets, performs data cleaning, validation, transformation, and aggregation using PySpark, and loads the processed analytical data into SQL Server. Apache Airflow automates the workflow, while Docker provides a consistent and reproducible execution environment.

Throughout this project, modern data engineering practices such as workflow orchestration, scalable data processing, data validation, and structured data storage were successfully applied. The generated analytical datasets can support business reporting and future data-driven decision making.

This project provided valuable practical experience in building production-like ETL pipelines and strengthened our understanding of real-world data engineering concepts and technologies.